

MODUS Enterprise AI Research Agent

10-15 Minute Live Demo Script + Exactly What to Show

Main demo query

How can AI improve supply chain forecasting in manufacturing?

Backup query: How can AI improve energy efficiency in manufacturing plants?

Time	Section	What to show
0:00-1:00	Problem + solution	App home screen; explain this is structured enterprise research, not a generic chatbot.
1:00-2:30	Architecture	README architecture diagram; show Streamlit -> Research Service -> Chroma/Tavily -> Groq -> SQLite.
2:30-5:30	Live query	Run the main query in deployed Streamlit and wait for the report.
5:30-7:00	Evidence + citations	Scroll to Supporting Evidence; show internal/web labels and clickable source URLs.
7:00-8:30	Reliability	Show confidence, citation IDs, weak-evidence behavior, retries/fallback concept.
8:30-9:30	Persistence	Show Research ID and Recent Research History; explain SQLite source/history persistence.
9:30-11:00	Code structure	Open research_service.py, retrieval_service.py, validation_service.py, repository.
11:00-12:30	Scale + limitations	Explain MVP vs production: PostgreSQL, managed vector DB, auth/RBAC, async ingestion.
12:30-13:30	AI disclosure	Show docs/AI_TOOL_DISCLOSURE.md and explain human review/testing.
13:30-15:00	Q&A; / backup	Use backup query or answer technical questions.

1. Opening - 0:00 to 1:00

SHOW: The deployed Streamlit home page before running anything.

SAY: "I built an Enterprise AI Research Agent for structured business research. The goal is not to behave like ChatGPT with web search. The system retrieves internal knowledge, searches external sources, stores evidence, validates citations, produces structured findings, and keeps traceability through research IDs and persisted sources."

2. Architecture - 1:00 to 2:30

SHOW: README.md -> System Architecture Mermaid diagram.

SAY: "The UI is Streamlit. The same business logic is reusable through FastAPI. A research request goes to the research orchestrator. Internal evidence is retrieved from Chroma using SentenceTransformer embeddings, while Tavily provides external research. I merge the evidence, assess its strength, then use Groq for grounded structured synthesis. Citation validation checks that the model only uses valid evidence IDs, and SQLite stores both research history and source records."

3. Live research test - 2:30 to 5:30

SHOW: Enter the main query in deployed Streamlit. Industry = manufacturing. Click Run Research.

SAY while it runs: "This is a fresh research question. I am not selecting a hard-coded answer. The backend dynamically retrieves internal evidence and external web evidence, merges them into one evidence package, and generates a report only from those sources."

4. Walk through the result - 5:30 to 7:00

SHOW: Research Summary -> Key Findings -> Recommended AI Opportunities -> Risks and Gaps.

SAY: "The output is structured rather than conversational. Each finding includes source IDs. Opportunities also include expected value and implementation difficulty. If the evidence supports risks, those are shown as well. The LLM is not allowed to invent citation IDs."

5. Supporting Evidence - 7:00 to 8:00

SHOW: Scroll to Supporting Evidence. Expand one internal source and one web source. Click one Open Source link.

SAY: "This is the traceability layer. S1 is internal knowledge. The other sources are dynamically collected web evidence. For web sources I store and display the source title, type, URL, and evidence text. An evaluator can inspect the exact evidence used to support a conclusion."

6. Confidence + guardrails - 8:00 to 9:00

SHOW: Evidence Confidence and Research ID.

SAY: "Confidence is based on the evidence quality rather than the LLM simply claiming confidence. Internal retrieval uses semantic distance. Web evidence is not given a fake Chroma distance. If the evidence is weak, the system returns a limitation instead of pretending the answer is strongly supported. I also validate every citation ID before returning the final response."

7. Persistence - 9:00 to 9:45

SHOW: Recent Research History; point to the current query and research ID.

SAY: "The application stores the research record in SQLite and also persists the supporting sources. I tested this by stopping Python, restarting it, and retrieving the source records using the same research ID. So the research trail is not only held in memory for one request."

8. Code walkthrough - 9:45 to 11:15

SHOW: [app/services/research_service.py](#) first. Then briefly show [retrieval_service.py](#), [validation_service.py](#), [repositories/research_repository.py](#).

SAY: "research_service.py is the orchestrator. retrieval_service.py handles semantic retrieval and duplicate removal. validation_service.py checks generated citation IDs. research_repository.py persists the research and supporting evidence. Keeping these responsibilities separate makes the application easier to test and replace individual components later."

9. Reliability choices - 11:15 to 12:00

SHOW: [llm_service.py](#) retry block if time permits.

SAY: "The Groq integration has bounded retries and specific rate-limit handling. During deployment testing I encountered free-tier token-per-minute limits, so I reduced unnecessary LLM calls and kept the primary generation path reliable. Auxiliary analysis can fall back without crashing the entire request."

10. Production scaling - 12:00 to 13:00

SHOW: [README Production Architecture / Scalability section](#).

SAY: "For the challenge I intentionally use SQLite and local Chroma because they are simple and free. In production I would move research metadata to PostgreSQL, use a shared managed vector database or pgvector, place documents in object storage, add asynchronous ingestion, authentication, RBAC, tenant isolation, rate limiting, and centralized observability."

11. AI tool disclosure - 13:00 to 13:40

SHOW: [docs/AI_TOOL_DISCLOSURE.md](#).

SAY: "I used AI coding assistance for architecture planning, code drafting, debugging, documentation, and preparation. I manually ran the project, tested the pipeline, debugged deployment issues, verified persistence and source traceability, and I can explain the components and design choices used in the final solution."

12. Close - 13:40 to 15:00

SHOW: [Return to the deployed Streamlit result](#).

SAY: "The key difference in this solution is that the answer is only one part of the system. The complete flow is question, evidence retrieval, external research, evidence storage, grounded synthesis, validation, and persistent traceability. A new research question can be entered and the same pipeline operates dynamically."

What to have open before the demo

- Tab 1: Deployed Streamlit application.
- Tab 2: GitHub README at the System Architecture section.
- Tab 3: research_service.py.
- Tab 4: retrieval_service.py and validation_service.py.
- Tab 5: research_repository.py.
- Tab 6: docs/DATA_MODEL.md.
- Tab 7: docs/TECHNOLOGY_LICENSES.md.
- Tab 8: docs/AI_TOOL_DISCLOSURE.md.

Do not spend time showing

- Every source file line-by-line.
- Long installation logs.
- Hugging Face unauthenticated warning - it is not a functional failure.
- API keys or Streamlit Secrets.
- Old failed deployment logs unless the evaluator asks about debugging.
- Every historical research entry.

Fast technical answers

Why Chroma?

Lightweight vector store for the MVP, supports semantic retrieval and metadata. In production I can swap it for pgvector or a managed vector database.

Why SentenceTransformers?

It gives local semantic embeddings without requiring another paid API for every retrieval operation.

Why Tavily?

It provides dynamic public web research so the evaluator can give a new question rather than relying only on hard-coded local documents.

How do you prevent hallucinated citations?

The LLM is only given explicit citation IDs, and a deterministic validator rejects IDs that are not in the retrieved evidence.

How is confidence calculated?

Internal semantic retrieval quality is measured using vector distance and evidence counts. External web evidence is handled separately rather than being assigned a fabricated internal similarity distance.

What if sources contradict?

The architecture includes contradiction analysis and the final prompt is designed to communicate uncertainty rather than silently choose one claim.

Why SQLite?

Fast, persistent, zero-infrastructure storage for the challenge MVP. PostgreSQL is the production migration path.

What was the hardest deployment issue?

Groq free-tier TPM limits when multiple LLM stages ran in one request. I added rate-limit handling and simplified the deployed path to prioritize reliable grounded synthesis.

Final checklist before recording / live validation

- ☐ Streamlit deployed app opens successfully.
- ☐ Run one fresh surprise query.
- ☐ Supporting Evidence shows internal/web source type.
- ☐ Web Open Source links are clickable.
- ☐ Research Confidence is displayed.
- ☐ Research ID is displayed.
- ☐ Recent Research History updates.
- ☐ .env and API keys are not visible or committed.
- ☐ README architecture diagram renders on GitHub.
- ☐ Keep the backup query ready if a provider temporarily rate-limits the first request.